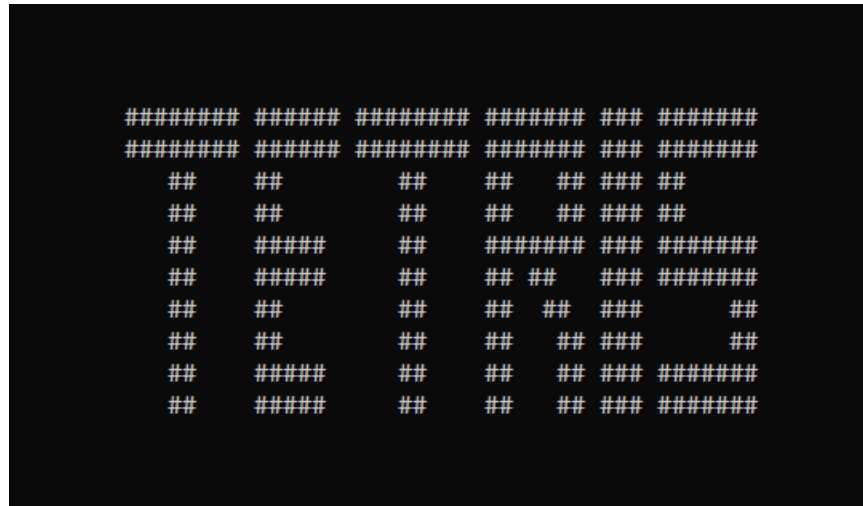


RELAZIONE PROGETTO DI PROGRAMMAZIONE

ANNO 2023/2024

Tetris versione ASCII usando la libreria curses



Autori e membri del gruppo:

- **Antonio Alati**
Email: antonio.alati@studio.unibo.it
Num. Matricola: 0001126541
- **Michele Palma**
Email: michele.palma2@stu.unibo.it
Num. Matricola: 0001142598
- **Michael Usai**
Email: michael.usai@studio.unibo.it
Num. Matricola: 0001114611
- **Manuele Lelli**
Email: manuele.elli@studio.unibo.it
Num. Matricola: 0001030993

TETRAMINI

Per costruire i tetramini, abbiamo valutato due approcci totalmente diversi:

- 1) Una matrice booleana.
- 2) Un'array di blocchi.

Matrice booleana

La matrice booleana avrebbe avuto dimensione $[D][D]$. In cui per D si intende la dimensione del quadrato di dimensione minima in cui il tetramino può essere inscritto (per esempio: il tetramino a forma di linea avrà dimensione 4, mentre quello a forma di quadrato avrà dimensione 2).

In questo caso, l'idea alla base è semplice, ogni valore dentro la matrice rappresenta un blocco del tetramino. Se il valore è true, allora quel blocco fa parte del tetramino, se invece è false no.

Tuttavia ciò porta ad una serie di svantaggi:

- I tetramini hanno dimensione D diverse.
- Quando bisogna ruotare o spostare il tetramino, bisogna iterare anche sui blocchi vuoti.

		X		
		0	1	2
Y	0	FALSE	TRUE	FALSE
	1	FALSE	TRUE	TRUE
	2	FALSE	TRUE	FALSE

Array di blocchi

L'array di blocchi (soluzione per cui abbiamo optato) ha dimensione $[N]$. In cui N è il numero di blocchi da cui è composto il tetramino. A differenza della matrice, in questa soluzione non memorizziamo quali blocchi sono presenti e quali invece non lo sono, ma andiamo direttamente a memorizzare le coordinate locali dei blocchi che compongono il tetramino.

Il "blocco" infatti non è nient'altro che una struttura che memorizza al suo interno 2 int, le coordinate locali X ed Y. Ciò ci permette di risolvere tutti i problemi che avevamo con la matrice booleana:

- Tutti i tetramini hanno 4 blocchi, a prescindere dalla loro forma. Quindi N è 4 per tutti i tetramini.
- Per ruotare e spostare il tetramino basta iterare 4 volte. Una per ogni blocco.

		X		
		0	1	2
Y	0		0	
	1		1	3
	2		2	

Blocchi[0].Y = 0; Blocchi[0].X = 1;
Blocchi[1].Y = 1; Blocchi[1].X = 1;
Blocchi[2].Y = 2; Blocchi[2].X = 1;
Blocchi[3].Y = 1; Blocchi[3].X = 2;

Oltre a ciò, ogni tetramino memorizza anche un parametro char che ha solo scopo estetico. Ed un parametro int che rappresenta la dimensione del quadrato di dimensione minima in cui il tetramino può essere inscritto (serve per l'algoritmo di rotazione del tetramino, e può variare da tetramino a tetramino).

SCHERMATA DI GIOCO

La schermata di gioco è composta da vari elementi: una griglia dove giocare, un timer, il punteggio, il livello, il numero di linee completate e il prossimo tetramino.

Griglia

Nella griglia vengono generati i tetramini. Tetramini che il giocatore può ruotare e spostare, ma che vengono anche abbassati dalla “gravità”, uno spostamento verso il basso del tetramino che effettua il sistema ciclicamente dopo un certo intervallo di tempo. Per ognuno di questi eventi bisogna gestire anche la collisione del tetramino controllato dal giocatore.

La griglia viene memorizzata da una matrice di char, in cui ogni char rappresenta un blocco. Se il char è un punto, allora significa che quelle coordinate nella griglia sono vuote, qualsiasi altro char significa che è presente un blocco in quella coordinata.

Collisione

Prima di ogni rotazione e spostamento del tetramino controllato dal giocatore il sistema controlla che non avvengano collisioni. In caso ci sia una collisione, l'azione viene annullata. Inoltre, per lo spostamento verso il basso (che sia l'input del giocatore o la gravità) in caso di collisione non solo annulla lo spostamento, ma “fissa” il tetramino in quella posizione e lo sovrascrive nella griglia. Successivamente genera anche un nuovo tetramino. Per collisione si intende:

- L'uscita di uno (o più) blocchi del tetramino dalla griglia.
- L'occupazione di uno (o più) blocchi già occupati nella griglia da parte del tetramino.

Movimento e stampa del Tetramino

In caso il movimento sia possibile, il sistema stamperà prima dei blocchi vuoti nei blocchi che occupava il tetramino prima dell'azione, poi ri-stamperà il tetramino dopo aver eseguito il suo movimento. Per la posizione del tetramino all'interno della griglia si usano 2 interi: *cx* e *cy* che mantengono traccia della posizione del tetramino.

Gestione degli Input del giocatore, e implementazione finale

Questa fase si sviluppa principalmente nelle funzioni della *classe tabella manageInp* e *start*. La prima prende in ingresso il tasto premuto dal giocatore e altri parametri per effettuare il movimento di un tetramino, mentre la seconda è la funzione chiamata per iniziare la partita.

Essa è composta da un ciclo while che termina quando viene raggiunta la condizione di GameOver, definita da una variabile booleana inizializzata a false che diventa true quando non è possibile posizionare il tetramino in cima alla griglia..

Punteggio e Livello

Per quanto riguarda il punteggio si ha un intero che parte da 0 a inizio gioco e viene incrementato di un punteggio base quando viene eliminata una riga più un bonus moltiplicato per il numero di righe eliminate contemporaneamente.

Per il livello invece si gestisce un intero che rappresenta il livello e un altro intero che rappresenta l'intervallo di tempo necessario a far cadere il tetramino corrente automaticamente. Praticamente ogni 10 righe eliminate il livello incrementa di uno e la differenza di tempo si accorcia del 10%.

MENU E LINKING

Per quanto riguarda i menù, abbiamo deciso di svilupparne 3:

- 1) Quello della schermata principale.
- 2) Quello che si apre con il game-over
- 3) Quello che permette il salvataggio del punteggio.

I primi due menù veri e propri sono strutturati in maniera identica: sono infatti due funzioni senza parametri ma che restituiscono un valore del quale verrà spiegata l'utilità più avanti. Abbiamo creato una struttura di char, necessaria per creare array in cui inserire le opzioni mostrate dai primi due menù.

Poi, in un while, abbiamo inserito il controllo del menù, previsto tramite l'utilizzo delle frecce e del tasto invio.

Tramite il tasto invio usciamo dal while e restituiamo l'indice dell'opzione selezionata in quel momento. Questo ci servirà nel main per la gestione delle azioni dei menù. La schermata di salvataggio del nome, invece, è una funzione void che prende come parametro un array di char sul quale verrà copiato il nickname inserito. Nella schermata è visualizzato il punteggio realizzato (viene passato dalla funzione di gioco) ed una zona in cui scrivere il proprio nickname (che deve ovviamente rispettare dei parametri di lunghezza e sintassi).

Premendo invio, quindi, il nickname viene copiato nell'array di char.

La parte del linking consiste nel mettere insieme tutti i pezzi fatti separatamente e creare il vero e proprio main, che ovviamente sarà la parte più breve perché tutto il codice è scritto nelle funzioni delle varie librerie.

Il main si sviluppa all'interno di un while che presenta le opzioni del menù principale (new game, leaderboard, quit) condizionato dalla flag "esc", falsa di default e che diventa true solamente nel momento in cui viene selezionata l'opzione "Quit".

CLASSIFICA

L'implementazione della classifica consiste di diverse funzioni per gestire un sistema di punteggi salvato in maniera permanente su un file esterno chiamato "score.txt" nel quale vengono salvate sotto richiesta, le principali informazioni riguardanti la partita, tra cui nickname, score e la durata.

Per quanto riguarda la scrittura e lettura su file ci siamo fatti affiancare dalla libreria "fstream" della quale abbiamo usato rispettivamente i tipi: *ofstream* per istanziare un oggetto per la scrittura su file e *ifstream*, per istanziare un oggetto dedicato alla lettura da file. Le funzioni dedicate alla visualizzazione della classifica su schermo sono quindi: *addplayer()*, *readFromFile()*, *insertsorted()* e *displayScores()*:

- **AddPlayer()**

Istanza un oggetto di tipo *ofstream* per scrivere su file in modalità append, poichè per semplicità, si è scelto di inserire i salvataggi nel file senza ordine, per poi successivamente, chiamare altre funzioni per leggere da file e ordinare la classifica durante la visualizzazione su schermo.

- **ReadFromFile()**

Istanza un oggetto di tipo *ifstream* per leggere i salvataggi dal file "score.txt" e inserirli nella lista concatenata usando la funzione "insertsorted()". Se il file non può essere aperto, viene stampato un messaggio di errore. La funzione legge tutti gli score dal file e li aggiunge alla lista nella posizione corretta.

- **InsertCode()**

Viene chiamata nel corpo della funzione "readFromFile()" ed inserisce un nuovo salvataggio nella lista concatenata in maniera ordinata. La lista è ordinata prima per punteggio decrescente, e in caso di parità di punteggio, per tempo crescente. Se la lista è vuota o il nuovo punteggio è maggiore del primo nodo, il nuovo nodo viene inserito in testa. Altrimenti, la funzione percorre la lista per trovare la posizione corretta per il nuovo nodo e lo inserisce mantenendo l'ordine.

- **DisplayScores()**

Ha il compito di stampare a schermo la lista concatenata di punteggi. Per visualizzare interamente la classifica, la funzione utilizza un offset che indica da quale elemento cominciare a stampare ad ogni esecuzione del ciclo while.

Per ordinare la classifica anche in base al tempo di esecuzione della partita (in caso di punteggi uguali di due player diversi), sono stati implementati due metodi della classe *tabella*, *getminutes()* e *getseconds()* i quali rispettivamente ritornano i minuti e i secondi trascorsi da quando è stata avviata la partita.